Replayable Tool Programs with Human Approval Boundaries

OpenBot

***Abstract*—A language model can write a short program to call tools, join their results, and return a small answer. Human approval makes this method harder: a program may stop before a write and start again much later. This design and analysis paper defines a restricted interpreter, an indexed call log, and an approval-aware replay protocol. A simple positional baseline reuses completed outcomes but misses some changes in the program environment. A stronger method fixes that environment, records complete call batches, and checks that replay consumes the required history. We prove local-state reconstruction at result-delivery boundaries under fixed local inputs and ordered history. We also give a limit: a tool log alone cannot resolve a send whose outside result is unknown. A finite executable model checks saved-outcome restarts, approval, denial, changed bindings, changed batches, and early return. A cost model separates model rounds and context size from tool work and repeated local work. These bounded checks are not a JavaScript interpreter benchmark or a measurement of model quality.**

***Index Terms*—Tool programs, Deterministic replay, Human approval, Language model agents**

I. Introduction

A useful agent may read ten records, select three, join fields, and send a report. Direct tool use puts each result into the model's conversation. A generated program can perform selection and joins locally, then return a small value. It also expresses loops and dependencies without a model call for each step.

This shift changes the unit that must pause for human control. A person may allow reads but want to inspect each write. Approval must name the concrete call and its arguments. It cannot act as a broad grant for every action that a program might later choose. A program host therefore needs a way to stop execution, retain its prior observations, and continue after a decision.

Keeping a live interpreter during a long wait is one option. Saving its entire memory is another. This paper studies a third method: keep the program and its tool-call history, then run the program again from its beginning. Completed calls supply saved outcomes. Local variables are rebuilt by execution. A waiting call proceeds only after its decision becomes known. The main question is when this method preserves the meaning of a program that stays alive across the same wait.

We define a positional replay baseline, then strengthen its environment checks, batch records, and completion rule. Our contributions are a frame-level transition system, a conditional reconstruction proof, an unknown-outcome limit, and a finite model with explicit weaker variants. We also bound the storage needed for exact replay and the local work repeated at approvals. Replay and durable approval are established ideas. Our claim is their precise composition for generated tool programs, rather than a new general workflow engine. We do not claim a deployed implementation of the full protocol.

The scope is program meaning and efficiency. We assume an outside execution service owns access checks, call outcomes, and safe handling of uncertain remote actions. We do not solve distributed owner takeover, credential delivery, or the choice of which actions deserve approval. Our claims concern interpreter inputs and avoided sends of completed calls.

II. Related Work

PAL expresses language-model reasoning as programs and delegates computation to an interpreter [1]. Program of Thoughts also separates language reasoning from numerical computation [2]. These methods support moving exact operations into code. Their stated tasks do not establish a recovery contract for programs that have already changed an outside system. We study that added contract, not improved arithmetic reasoning.

CodeAct uses executable Python as an agent action space and supports control flow, several actions, and correction across interactions [3]. Our proposed interpreter instead exposes only named tool bindings and deterministic local computation. The question is which observations and event orders must be restored when a generated program crosses an approval boundary. CodeAct's reported task results do not settle that question for this protocol.

R2 records selected function calls and replays their results at a chosen application interface [4]. It explains why that interface must enclose relevant nondeterminism and preserve event order. A tool binding plays a similar role here. The restricted interpreter makes the boundary smaller than a general application interface, while approval permits the recorded trace to grow after a wait.

Beldi provides fault-tolerant and transactional serverless workflows through a library and runtime [5]. Its guarantees depend on its data structures and execution protocols. A tool-result log alone cannot import those guarantees for arbitrary remote tools. We therefore separate replay of a known result from recovery of an outside action whose outcome is unknown.

Temporal requires workflow code to produce commands compatible with its history. Its documentation covers deterministic code, time, randomness, and code changes [6]. It also documents approval through a signal recorded in workflow history [7]. Our analysis specializes these concerns to generated programs, complete batches of promise results, and the repeated cost of rebuilding local state. Temporal already com-

pares replayed commands with saved event history and rejects mismatches [6]. Our deliberately weak positional baseline does not stand for Temporal, R2, or Beldi. The proposed frame check is a sufficient rule for this restricted interpreter, not evidence that mature workflow engines lack replay checks. A future performance comparison must include such an engine, as well as the simple baselines.

III. Interpreter and Call Model

A program P is the body of an async function in a restricted JavaScript language. It has arrays, plain objects, pure local operations, promises, exceptions, and three host functions: `call(name, args)`, `now()`, and `random()`. The host gives a read-only `bindings` object that describes available tool names. The program has no network, files, imports, native timers, shared memory, or other clock. The restriction is part of the design, not a claim about unrestricted JavaScript. In particular, locale-sensitive ordering, native randomness, stack inspection, weak references, finalizers, and unrecorded cancellation are outside the language. A host must reject these features or give them fixed semantics; an API allowlist alone does not prove that this boundary holds.

`call` accepts a tool name and a plain JSON object. JSON contains arrays, objects, strings, finite numbers, booleans, and null. Accessors, functions, symbols, cycles, and other values are rejected. A valid call returns a promise. A successful tool result resolves it with a copied JSON value. A failed result rejects it with a fixed error record. A denial uses the error name `ApprovalDenied`. A request for additional input uses `MoreInput`; an outside controller must answer it before a new program can use that input.

Let B be the binding map, t the saved time, and s the saved random seed. A binding maps a name to a tool identity and argument schema. Every `now()` returns t. For a completely specified illustrative generator, treat s as an unsigned 32-bit integer. Each `random()` sets s to $(1664525s + 1013904223) \bmod 2^{32}$ and returns $\frac{s}{2^{32}}$. This generator is for repeatability, not secret keys. A run record saves the initial seed; each attempt resets to it.

The interpreter state L includes variables, the pending promise queue, the next call index, and generator state. Valid calls receive successive indices starting at zero. A synchronous call to a missing name raises `BindingUnavailable` before an index is allocated. The interpreter executes ordinary code and queued jobs until it returns, fails, exceeds its limits, or needs tool outcomes. At this last point it hands the host the ordered list of calls issued since the previous handoff. We call that list a batch.

A call descriptor contains its index i, name b, and encoded arguments x. Arguments are copied and encoded at issuance, before later program steps can mutate their objects. A digest $h = H(b, x)$ is a compact value derived from the name and arguments. The proof assumes no digest collision among calls in the run. The host also retains the full arguments because a person must inspect them.

Three orders matter. Issue order assigns indices when the program calls bindings. Completion order describes when remote tools finish. Settlement order describes when the interpreter resolves or rejects their promises. Only settlement directly schedules promise continuations. Our host waits for a batch's needed results, then settles them in index order before running more promise jobs. Tool completion speed therefore does not select the visible order within that batch.

IV. Defined Positional Replay Baseline

The baseline stores P, t, the initial s, and a partial call log indexed by i. Its binding map is rebuilt from current configuration on every attempt. It has no saved map fingerprint or runtime version. For argument comparison it serializes JSON objects in their construction order. It does not sort keys. Thus two objects with the same fields but different insertion order can have different digests.

Each logged entry contains the name, digest, full arguments, and an outcome. The outcome is `Waiting`, `Denied`, or `Done`. A done entry includes the exact success value or failure record. A waiting entry names a decision or an outside invocation that has not yielded a final outcome. An immutable decision record names the run, index, binding name, and argument digest. It retains the full reviewed arguments. Decisions are durable inputs, not grants for the whole program.

For each batch, the host inspects calls in index order. It first checks that the current name exists and the arguments are valid. If an entry exists at that index, a different name or digest stops the attempt with `ReplayMismatch`. A matching ended entry goes into the reply list and never enters the send list. For a waiting entry, the host reads its decision and invocation record. A known outcome becomes ended; an open decision or unknown outside outcome suspends processing.

For a new call, the host saves a waiting entry before allowing a send. If its decision is already allow, it joins the send list. Otherwise the host stops inspecting the batch at that call. Earlier allowed calls may still run, with a fixed concurrency bound c. Their results are saved. If any decision remains open, the worker is discarded and no result from this batch is delivered to it. Later issued calls in that batch have no log entries yet.

If the batch can complete, the host saves all outcomes and returns them in index order. Failure values and denials count as ended outcomes, so the program may catch them. An invocation with an uncertain outside outcome stays waiting. Only the outside execution service may resolve that uncertainty or permit another attempt. A storage error produces a failed run, not a request for human approval.

On resume, the host creates a new interpreter from P, t, and the saved initial seed. It applies these same per-call rules. On ordinary return, the baseline accepts the value without checking whether the old log has unvisited entries. It also does not reject an otherwise completed program merely because it issued an unawaited tool promise. These completion rules define a weak baseline, rather than an endorsement for use.

TABLE I
THE POSITIONAL BASELINE. NO EMPIRICAL SYSTEM IS ASSERTED TO USE THESE RULES.

Case	Condition	Baseline action
Replay	Matching ended entry	Queue saved value or error
Mismatch	Same index, other name or digest	Stop before this send
New call	Valid and allowed	Save waiting entry, execute, save result
Wait	Decision open	Keep earlier results; discard worker
Return	Program ends	Accept without checking unused log

A. What batching changes

With $c = 8$, up to eight allowed requests can run together. A slow request can hold back the entire batch. `Promise.all` over independent reads fits this protocol. In contrast, `Promise.race` over two direct tool promises in one batch does not reliably choose the fastest remote service. Both results wait at the barrier; the earlier index is settled first. With immediate handlers it can win even if its remote tool finished last.

Suppose a batch issues calls zero, one, and two. Zero is allowed and one waits for approval. The host may execute zero but does not inspect or send two. After approval, replay recreates all three indices. Do not deliver zero's cached result before that batch is recreated. Early settlement could run its continuation and change which calls belong to the batch. An approval boundary is also not a transaction: the completed action at zero remains completed if one is later denied.

V. STRONGER REPLAY PROTOCOL

We propose three additions: an environment manifest, a complete batch history, and a strict completion rule. A manifest is a saved description of the local execution environment. It contains the program digest, interpreter and helper versions, generator definition and seed, clock value, encoder version, resource settings, and the complete binding map exposed to the program. Each binding includes its stable identity and schema revision. Matching a display name alone is insufficient.

Resume requires exact manifest equality or continued use of the saved environment. Any inability to provide it stops execution before a new outside send. Version migration is outside this protocol. Current access checks still apply to new sends; a fixed binding identity is not a permanent grant. The manifest cannot freeze the outside world.

The stronger encoder uses the JSON Canonicalization Scheme in RFC 8785: object keys use UTF-16 code-unit order, arrays keep their order, numbers use its fixed binary64 encoding, and strings keep their Unicode values [8]. Duplicate keys, lone surrogates, and non-finite numbers are rejected before recording. Negative zero becomes zero. A Python key-sort test with ASCII names is only a check of one consequence of this rule, not a test of full JCS conformance. Validation and normalization occur before the first delivery of data to the program, as well as before hashing. Otherwise replay might return an object with a different key order even when the digest matches. All recorded results and error records are immutable and delivered as fresh copies with the same normalized form.

The history now has batch records as well as call outcomes. Before processing any new batch, the host saves its complete ordered descriptors, including calls after the first approval boundary. This record is an issue frame. A delivery frame records the complete ordered outcomes later given to the interpreter. An issue frame may remain open across many approvals. An open frame has no delivery frame, even if some of its calls already have saved outcomes.

During replay, the interpreter must recreate the next issue frame in full before it receives any of that frame's outcomes. The host compares its length, indices, binding identities, and digests. A mismatch ends the run. A previously delivered frame then supplies exactly its saved outcome vector, with no outside sends. For an open frame, the host reuses ended outcomes and continues the decision-and-send rules from the baseline. It stops at the first unresolved decision and retains the whole frame. Once every member ends, it saves a delivery frame and settles all promises in index order.

The host may append a new frame only after all earlier required frames have been matched and delivered. It never treats an unexpected call in a recorded frame as a new suffix. Before normal return, every required recorded frame must have been consumed and no issued tool promise may remain pending. A failed computation is reported as failure rather than completion. Resource exhaustion is not silently reclassified as a replay mismatch.

```
require the saved local environment
start the program with saved time and seed
when the interpreter hands over a batch:
  if a required issue frame exists:
    require an exact match of the entire batch
  else:
    save the entire new issue frame
  if its delivery frame exists:
    settle the saved vector in recorded order
  else:
    reuse ended outcomes without sending
    process remaining decisions in index order
    save each allowed call's outcome
    if any call still waits: discard interpreter
    otherwise save and settle the complete vector
on normal return:
  require all required frames consumed
  require no pending tool promises
```

Saving an outcome precedes its delivery. A crash between those actions may cause local computation to repeat, but the known outcome remains available. A crash between a remote send and outcome storage remains the outside execution service's problem. The protocol does not turn that uncertain action into a known completed one. The host processes each run under one current owner, or uses an equivalent atomic claim on each call. Without that condition, two workers can both observe a missing outcome and send the same call. Neither a manifest nor a frame comparison prevents that race.

A. Durable state and transition rules

Write the durable state as (M, I, O, D): a manifest M, issue frames I, final call outcomes O, and delivery frames D. Calls use the pair `(run, index)` as their identity.

A descriptor includes the stable binding identity and the normalized argument bytes. A delivery frame is an immutable reference to one final outcome for every descriptor in its issue frame. The replay cursor k belongs to the current interpreter attempt and starts at zero; it is not a cursor shared between workers.

Three host transitions change this durable state; a fourth changes the current interpreter attempt. **Issue** appends a full frame after all earlier frames have been consumed by this attempt. **End** inserts a final success, failure, or denial for a call; it never replaces one final value with another. **Seal** creates a delivery frame once all its outcomes exist. **Deliver** reads that saved frame, settles its promises, and advances the local cursor. A replayed Issue checks equality instead of appending. A replayed End reads the saved outcome instead of sending. A replayed Seal reads the existing frame. Deliver may repeat in a new interpreter, but it performs no outside action.

These rules require atomic insertion, durable reads, and no deletion of required history. An open frame can have an empty or partly filled outcome set. It has no delivery frame. The invariant is that every delivered value names a final saved outcome in a complete matched frame. Approval is an input to End, not a direct state update by generated code. A decision for another run, index, binding identity, or argument value cannot enable this End transition.

VI. Conditions and Proof Arguments

We compare replay with an uninterrupted logical interpreter that retains its state while waiting. It follows the same batch rules and receives the same decisions and tool outcomes. This reference execution is defined by those inputs; it is not the abandoned worker's nonexistent future. Equality is claimed at delivered frame boundaries. We do not claim that a newly restarted worker immediately equals the transient state of a discarded worker with partly processed requests.

Five assumptions govern the result, in addition to the single-owner or atomic-call condition above. First, program and local environment are fixed as defined by the manifest. Second, all inputs that affect control flow are deterministic from initial data or restored from history. Third, the interpreter's job order and host settlement rules are fixed. Fourth, stored frames and outcomes are complete, uncorrupted, and durably available before they are delivered. Fifth, both executions have enough resources to reach the compared boundary. Fixed clocks and random seeds help meet these assumptions but do not prove that no other input exists.

Claim 1: state reconstruction. Under these assumptions, replay and the reference interpreter have equal local state after each finite delivered frame. Local state includes variables, generator state, next call index, and queued jobs, up to a consistent renaming of allocated objects. The boundary is after ordered settlement but before queued jobs run. The proof is by induction. Initial states agree. Deterministic local steps preserve agreement and issue the same next batch. The complete frame check prevents outcomes from being released to a different batch. Equal normalized outcomes settled in equal order schedule equal jobs. Therefore the resulting states agree. At an open frame, later decisions supply new inputs to both executions. Once it is delivered, the same induction step applies.

Claim 2: no resend of matched ended entries. An ended outcome is copied into a reply vector before the send rule is considered for that call. It cannot enter the send list. Thus replay causes zero outside sends for that known ended entry. Summing over any matched ended prefix preserves zero. This is narrower than exactly-once execution: it says nothing about a remote action whose result was never durably recorded.

Claim 3: encountered divergence is detected. If a reproduced descriptor differs from its saved descriptor in index, binding identity, or encoded arguments, the complete frame check rejects it before sends from that frame. The claim assumes no digest collision. The positional baseline proves only the analogous per-call property for names and arguments at encountered indices. It does not detect all changes in binding meaning, skipped calls, or batch shape.

Claim 4: required-history consumption. In the stronger protocol, a normal return with any required frame left unread is rejected. This follows directly from the completion rule. It closes the early-return counterexample below. It does not prove arbitrary program equivalence. Equal traces can still produce different local results if a proof assumption is broken, and an explicit failure need not consume a successful history.

Claim 5: conditional progress. Suppose the reference program reaches a normal return after a finite number of frames. Suppose every open decision and unknown invocation eventually has a final outcome, every enabled storage step eventually runs, and only finitely many crashes occur. The required environment and history must remain available, and a crash-free attempt must have enough time and memory for the full rebuilt prefix. Then replay eventually reaches the same return. After the last crash, finite local computation reaches each next frame; its finite set of outcomes eventually permits Seal and Deliver. Induction over the remaining frames proves the result. An infinite approval wait or a replay prefix longer than every worker time limit breaks the premise, even if each remote action has ended.

Claim 6: unknown outcomes cannot be settled by local replay alone. Consider two executions with the same durable Waiting record. In one, a write changed the outside system and the worker died before saving the receipt. In the other, the worker died before the write reached that system. If the outside service offers no query, deduplication key, or transaction shared with the host, both executions give the host the same evidence. A resend can repeat the first effect; no resend can omit the second. Any fixed choice therefore loses either no-repeat safety or eventual execution in one case. Random choice cannot give a certain guarantee in both. This is an indistinguishability argument about the stated interface, not a limit on richer remote protocols. RPC systems have long treated lost replies and duplicate suppression as transport and server concerns [9].

Refinement boundary. Remove replay-only local steps and repeated Deliver events from an attempt trace. The remaining new issue frames and new final outcomes extend the same reference history under Claims 1–4. This projection is the

refinement claim: repeated local work does not create a new logical tool call. It does not equate real-time completion orders, make multiple tools atomic, or repair a wrong generated program. Actual duplicate-effect freedom still depends on the outside execution service in Claim 6.

These arguments rely on data-flow equality, not on the world remaining unchanged. A read saved before approval remains the old read even if its remote record changes. That choice preserves the reviewed arguments. If the task requires fresh data, it needs a later explicit read and any decision required for the resulting write. Silent refresh during replay would invalidate the common-input premise.

VII. Worked Program and Counterexamples

Consider two notes and a board. `notes.read` returns a JSON object with `title` and `share`. `board.write` records its argument object and returns `{ok: true}`. Reads are allowed; each board write needs a decision. These bindings are synthetic.

```
const [a, b] = await Promise.all([
  call("notes.read", { id: "one" }),
  call("notes.read", { id: "two" }),
])
const titles = [a, b]
  .filter(x => x.share)
  .map(x => x.title)
try {
  await call("board.write", {
    text: titles.join("; "), time: now(),
  })
  return { sent: titles.length }
} catch (error) {
  if (error.name === "ApprovalDenied")
    return { sent: 0, denied: true }
  throw error
}
```

Set saved time to 1000. Note one is `{title: "Plan", share: true}`; note two is `{title: "Draft", share: false}`. Calls zero and one form the first frame. Call two forms the second frame with `{text: "Plan", time: 1000}`. Approval names this exact write. On allow, replay rebuilds `titles` from saved reads and sends the write. On deny, the promise rejects and the catch block returns `{sent: 0, denied: true}`. The completed reads are not sent again in either continuation.

TABLE II
Alternative continuations of the same illustrative prefix.

Attempt	Index	History state	Outside action
First	0, 1	Delivered	Read two notes
First	2	Open	Ask for decision
Allow	0, 1	Replay frame	No read
Allow	2	Delivered	Write Plan
Deny	2	Delivered error	No write

In the failure-free allow trace there are three logical calls and three outside sends. A restarted plain script would repeat both reads unless it adds its own replay mechanism. If note one's title changes during approval, that script can compute different write arguments. Our reference instead retains the old read and the reviewed write.

For positional divergence, let a program read a note only when `bindings.extra` exists, then write a fixed report. Initially the read has index zero and the write index one. Remove `extra` during approval. The baseline rebuilds its binding map, skips the read, and encounters the write at zero. Its name differs from the logged read, so it returns `ReplayMismatch` before that write. This conclusion follows from the baseline's index comparison.

For missed divergence, change the branch to `if (!bindings.extra) return 0`. Before removal the program reaches the waiting write. After removal it returns without encountering any old call. The baseline accepts zero because it has no required-history check. The stronger design rejects the changed manifest before execution. Even if an omitted environment input caused the same early return, its terminal consumption check would reject success with unread frames.

A third case changes the target tool behind an unchanged name. The baseline's name and argument digest still match; it can reuse an old read or send a newly approved call through the new mapping. The stronger manifest rejects the changed binding identity. This does not claim to detect a tool provider silently breaking its own stable contract; that is outside the manifest's authority.

VIII. Cost and Model-Round Analysis

Let n be logical tool calls and d dependent tool stages. Independent calls can share a stage. Let a be approval suspensions, R model-round time, E tool execution time under the chosen concurrency plan, and W human waiting time. Keep W separate because code does not make a person decide faster.

A serial direct-call agent uses about n action rounds. A strong direct-call baseline batches independent calls and can use about d. Both counts exclude a common final answer. A program needs generation and perhaps declaration lookup; let their total be g. Let r be repair rounds for invalid or incorrect code. The model-time difference against the strong baseline is approximately $(d - g - r)R$. A negative value favors direct calls. These are counts, not measured latency.

Code need not reduce the n tool requests. It can reduce model input. Let S be total tool-result tokens exposed by direct calls, K the program's returned summary, A declarations, and G generated code. First-use exposure compares S with $K + A + G$, excluding common prompts. Repeated conversation prompts can amplify large results. Caching and compaction must be measured rather than assumed absent.

Let C be local work in one complete execution and C_j the rebuilt prefix before approval j. Without unrelated crashes or repairs, restart-from-top work is

$$C_{\text{total}} = C + \sum_{j=1}^{a} C_j. \tag{1}$$

If each prefix costs at most C, then $C_{\text{total}} \le (a+1)C$. For a equal-cost segments before successive approvals, each of cost u, the repeated prefix work is $u\frac{a(a+1)}{2}$. Additional final-segment work is separate. Many approval boundaries

can therefore make state checkpoints or smaller programs preferable.

For an illustrative token calculation, take twelve calls in three stages. Each returns 8000 tokens. Suppose the program returns 600 tokens, reads 1400 declaration tokens, and emits 2000 code tokens. Exposure is $12 \times 8000 = 96000$ versus $600 + 1400 + 2000 = 4000$ tokens. The difference is 92000 under these assumptions. With two lookup-and-generation rounds, however, the strong direct baseline offers only one avoided action round before repairs. Comparing only against twelve serial rounds would overstate the opportunity.

The full cost adds interpreter startup, normalization, trace writes, repeated local work, and result delivery. The stronger protocol also saves complete issue and delivery frames. Human decision notices may cause more model calls in the surrounding conversation. Small results, shallow tasks, or frequent repairs can erase the saving. A task with large data and a known compact projection is a better candidate, subject to correct results and writes.

A. History space and work bounds

Let X be the bytes of normalized call arguments and final outcomes, n the call count, f the frame count, and M the manifest size. When delivery frames refer to outcomes rather than copying them, stored history is $O(X + n + f + M)$ for fixed-size identifiers. A replay attempt reads this history once in frame order. It also pays the program's local work and the cost of comparing argument bytes. Sorting an object with q keys costs $O(q \log q)$ comparisons; the total normalization cost also includes reading all value bytes. Thus a count of tool sends alone is not a bound on replay CPU or storage.

There is a simple lower bound on exact replay. A completed read may return any of 2^b independent bit strings, and a permitted later program may return that string unchanged. If neither the interpreter state nor the outside value can be recovered, the durable record must distinguish those 2^b cases. A fixed-length record needs at least b bits. A digest smaller than b cannot reconstruct every possible result. Compression can help a chosen workload, and remote content-addressed storage can move the bytes, but neither removes this worst-case information requirement. The protocol must retain outcomes or an equally reliable source until no permitted replay needs them.

IX. Executed Model Checks

The accompanying experiment artifact implements a finite frame model. It separates the send list from the saved outcome map and matches complete frame vectors. It is a finite transition model with caller-supplied frames; it does not parse JavaScript, run promises, model a database, or prove sandbox isolation. In particular, it cannot by itself test Claim 1′s queue-state equality. The model tests consequences of the protocol at its host boundary.

The run uses 30 seeds, numbered zero through 29. The artifact records checks, seed counts, limitations, and source hashes in a JSON report. The fixed examples retain the reviewed `Plan` value after the outside value changes, send no write on denial, reject a changed manifest, reject a split batch, and reject an early return with unread history. Thirty seeded read-call plans use 4–24 calls and batch widths 1, 4, or 8. One chosen stop per plan checks that resumed values equal saved values. An expanded sweep starts a separate trial at every saved-outcome cut in these plans: 408 cuts in total. Each trial compares frame replay with a plain restart that sends the completed prefix again. The report for this configuration has 1,780 assertions; all pass.

TABLE III
Executed model checks with 30 seeds. The first three rows use plain restart as the weaker variant. The last row uses positional replay. Send counts are model events, not HTTP requests or measured time.

Finite model measure	Weaker variant	Frame protocol
Saved-outcome cuts	408	408
Total synthetic sends	9,939	6,490
Repeated completed sends	3,449	0
Changed replays rejected	0 / 4	4 / 4

The four changed replays alter the manifest, split a frame, merge frames, or return before consuming history. The positional model accepts all four. The stronger model rejects all four and adds no sends during rejection. These are chosen counterexamples, not a random estimate of mismatch-detection accuracy. They test different checks together; they do not isolate all possible interactions among the checks.

A separate two-world fixture gives the host the same empty receipt map after either zero or one outside effects. A blind retry yields one or two total effects; holding the call yields zero or one. This executable example checks the finite witness for Claim 6. It does not resolve the unknown call, and its outcomes are not included in the saved-outcome no-repeat count.

These are deliberately bounded observations. A stop after a remote effect but before outcome storage is excluded from the no-resend result, as required by Claim 6. The checks do not measure completion-versus-settlement order, runtime versions, JCS Unicode cases, model generation, repair cost, latency, or token use. Their assertion count is a measure of the exercised model, not an estimate of field reliability. The full controlled study below remains unrun.

X. Proposed System Evaluation

No interpreter benchmark or model-generation evaluation has been run for this paper. We propose a deterministic mock-tool harness with selection, joins, pagination, bounded loops, caught failures, and writes based on earlier reads. Each task has an oracle answer and expected ordered writes. Control data size, tool delay, and decision delay separately.

Compare direct calls with parallel-call support, plain scripts restarted after decisions, the positional replay baseline, and the stronger frame protocol. Add a plain-script variant that retains its process during approval. Give each variant equal schemas, decision rules, memory limits, model families, and result limits. Every new write still needs the same decision. A restarted script must obtain a new decision if fresh reads change its arguments.

Use dependent stages of one, three, and six; batch widths of one, four, and sixteen; and result sizes of one, sixteen, and 128 KiB. Approval patterns are none, one middle write, and every write. Vary allow and deny decisions independently. Pair variants on task, data seed, and delay schedule. Reserve held-out task seeds for final reporting.

Separate fixed-program and model-generation tracks. In the first, all runtimes receive the same hand-checked program. This isolates replay and batching. In the second, a pinned model writes the program or selects direct calls. Count lookup, repairs, final explanations, failures, and timeouts. Restricting token reports to successful programs would hide representation costs.

Correctness measures are final-value agreement, ordered-write agreement, sends without the required decision, repeated sends of known ended calls, and the first mismatch position. Record issue, completion, and settlement order independently. Perturb binding lists, binding targets, argument construction order, promise-handler order, and early return. Failure labels should distinguish bad generated logic, violated replay assumptions, expected mismatch rejection, and uncertain outside outcomes.

Efficiency measures are model calls, token usage, tool sends, replay hits, trace bytes, interpreter CPU time, peak memory, and total latency. Report human waiting separately. If money is reported, use recorded usage and an explicit dated price table. Retain token and resource counts so findings remain useful when rates change.

Ablations remove fixed time, seeded randomness, full-frame matching, normalized encoding, manifest checks, and terminal consumption checks one at a time. Run these weakened variants only in the mock harness. Compare $c = 1$ with $c = 8$ to measure concurrency and the batch barrier. Include an early-delivery variant that settles cached results before the whole historical batch is matched; its traces test the reason for the frame rule.

Plan at least thirty task seeds per workload cell and five independent model samples per task in the generation track. Pair comparisons by task and delay seed. Bootstrap whole tasks, retaining their repeated samples, for confidence intervals. Report medians and upper latency quantiles, correctness proportions, and paired token differences. Publish failures as well as successes. A lower token average supports an efficiency claim only alongside preserved answer and write correctness.

XI. Reproducible Fixtures and Proof Obligations

A minimal harness can be built from the three synthetic tools already defined: note read, board write, and an optional extra binding. Store every board invocation in an ordered append-only list. Its oracle is exact argument equality, not a language-model judgment. Use time 1000 and seed 7. Reset tool data and write lists before each trial. Delay the first read by three scheduler ticks and the second by one tick. The completion order is then reversed while settlement remains indexed.

For the allow fixture, pause before call two, change note one's title to `Changed`, allow the original write, and resume. The expected write still contains `Plan` and time 1000. The expected read-send count remains two under either replay protocol. For denial, expect an empty write list and `{sent: 0, denied: true}`. These are predicted outcomes from the specified rules, not reported test results.

For mismatch fixtures, run the conditional-read and early-return programs above, remove the extra binding, and resume. Record the baseline's different outcomes and the stronger manifest rejection. For the identity fixture, retain the name and change its target identity. For the key-order fixture, construct the same two-field object in reversed insertion order; the baseline digest changes while the stronger normalized encoding agrees. Report these mechanisms separately.

Inject a stop after issue-frame storage, after outcome storage, and before delivery-frame storage. A saved ended outcome must never be sent again. Also inject a stop after remote execution but before outcome storage. The harness must mark that outcome unknown and await the execution service; it must not count a fabricated success as replay correctness. This separates no resend of known outcomes from exactly-once claims.

A reproducible artifact should include interpreter and helper versions, complete manifests, generator definitions, programs, normalized inputs, decision sequences, fault schedules, resource settings, and all trace frames. Check the proof obligations directly: matching frame lengths; stable descriptors; outcome storage before delivery; no early settlement; no new frame before history consumption; and no successful return with required frames or pending tool promises. These checks follow directly from the algorithm.

XII. Limits and Conclusion

Fixed time is run time, not current wall time. Seeded randomness is repeatable, not fresh entropy on resume. Saved reads can become stale during approval. Those choices preserve a common input history but may not suit every task. A requirement for fresh data must be expressed as new work with its own concrete arguments.

The manifest fixes declared local inputs; it cannot prove that all hidden inputs were excluded or that a remote provider preserves its contract. Resource failures or unanswered decisions can prevent progress. Complete-frame checks and terminal consumption reject specific divergences without deciding general program equivalence. Remote side effects still require a separate recovery contract.

The finite checks support the specified host rules within their tested inputs. They do not establish a speedup, a JavaScript implementation theorem, or general field reliability. Replayable tool programs can keep intermediate data out of model context while retaining a concrete human approval boundary. Their useful contract is precise: stable local computation, saved tool observations, complete visible result frames, and no resend of matched ended calls. The defined baseline shows why positional matching alone is weaker. The method, proofs, and fixtures make the stronger contract testable.

References

[1] L. Gao *et al.*, "PAL: Program-aided Language Models," in *Proceedings of the 40th International Conference on Machine Learning*, in Proceedings of Machine Learning Research, vol. 202. PMLR, 2023, pp. 10764–10799. [Online]. Available: https://proceedings.mlr.press/v202/gao23f.html

[2] W. Chen, X. Ma, X. Wang, and W. W. Cohen, "Program of Thoughts Prompting: Disentangling Computation from Reasoning for Numerical Reasoning Tasks," *Transactions on Machine Learning Research*, 2023, [Online]. Available: https://arxiv.org/abs/2211.12588

[3] X. Wang *et al.*, "Executable Code Actions Elicit Better LLM Agents," in *Proceedings of the 41st International Conference on Machine Learning*, in Proceedings of Machine Learning Research, vol. 235. PMLR, 2024, pp. 50208–50232. [Online]. Available: https://proceedings.mlr.press/v235/wang24h.html

[4] Z. Guo *et al.*, "R2: An Application-Level Kernel for Record and Replay," in *8th USENIX Symposium on Operating Systems Design and Implementation*, USENIX Association, 2008, pp. 193–208. [Online]. Available: https://www.usenix.org/legacy/events/osdi08/tech/full_papers/guo/guo.pdf

[5] H. Zhang, A. Cardoza, P. B. Chen, S. Angel, and V. Liu, "Fault-tolerant and transactional stateful serverless workflows," in *14th USENIX Symposium on Operating Systems Design and Implementation*, USENIX Association, 2020, pp. 1187–1204. [Online]. Available: https://www.usenix.org/conference/osdi20/presentation/zhang-haoran

[6] Temporal Technologies Inc., "Temporal Workflow Definition." Accessed: Oct. 27, 2026. [Online]. Available: https://docs.temporal.io/workflow-definition

[7] Temporal Technologies Inc., "Approval Pattern." Accessed: Oct. 27, 2026. [Online]. Available: https://docs.temporal.io/design-patterns/approval

[8] A. Rundgren, B. Jordan, and S. Erdtman, "JSON Canonicalization Scheme (JCS)," technical report 8785, 2020. doi: 10.17487/RFC8785.

[9] A. D. Birrell and B. J. Nelson, "Implementing Remote Procedure Calls," *ACM Transactions on Computer Systems*, vol. 2, no. 1, pp. 39–59, 1984, doi: 10.1145/2080.357392.